Appendix K
Algorithms

The following is a list of Algorithms with the ability to interact automatically with each other to provide a complete controller. Depending on the combination of these algorithms and their specific use, controllers different in function and complexity can be configured.

PIDN	incremental real PID
PIDX	incremental ideal PID
PV	PV and SP limits, mode, initialization, tracking, etc.
OP	PID-OP value, limits, mode, initialization, etc.
MODE	MODE handling between PID?, PV, OP, ALARM, etc.
STATUS	Controller Status
ALARM	Controller Alarm

The following is a list of Stand Alone Algorithms which can be used in any combination:

LEADLAG	First Order LEAD and LAG
SUM	Summer
MUL	Multiplier
RATIO	Ratio and Bias
PROP	incremental Proportional with Gain
INT	Integral
DERN	real Derivative
DERX	ideal Derivative
LAG	First Order LAG
NOISE	Random Noise with Gain
DISTURB	General Disturbance Generator
SINE	Sine wave generator
SNDORDER	2nd Order System with variable damping
HILIM	High Limit
LOLIM	Low Limit
ROCHILIM	Rate of Change Limit High
ROCLOLIM	Rate of Change Limit Low
ILIM	Integral Limits
HIAL	High Alarm
LOAL	Low Alarm
ROCHIAL	Rate of Change Alarm High
ROCLOAL	Rate of Change Alarm Low
FREQTOTC	Frequency to Time Constant Conversion
EUTOPCT	Engineering Units to % conversion
PCTTOEU	% to Engineering Units conversion
LINK	Link if a status variable is true
MATH	general Mathematical
DIV	Division
HEATCOMP	Heat Balance Calc for Feed Forward Control
HEATSIM	Heat as function of fuel consumption for Simulation
MASSFLOW	Mass-Flow Compensation)
DEADTIME	Dead Time for Simulation and / or Control)

K.1 Interacting PID-Algorithm-Blocks to Build PID-Controllers

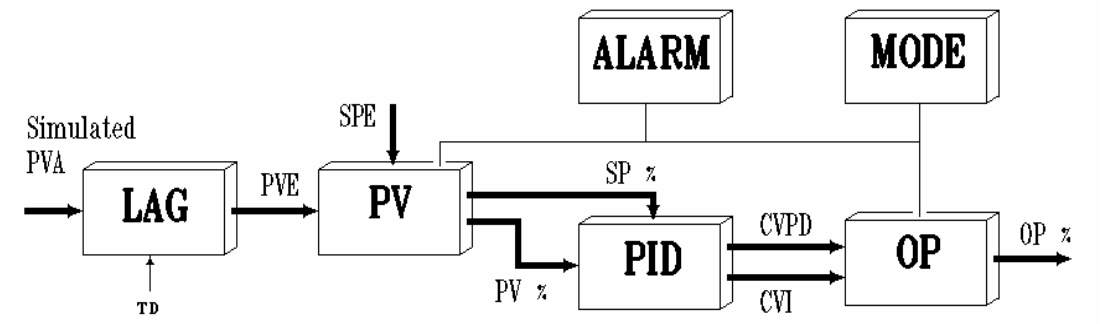


Figure K.1
Principle Interaction between Algorithms of a PID Controller

K.1.1 Configuration Example

Level Controller Algorithms and Variables

*ALGO1:LAG	P1=14	P2=46	P3=10;	
*ALGO2:PV	P1=14	P2=15	P3=4	P4=2
	P5=3	P6=1	P7=8	P8=1
	P9=5	P10=6	P11=24	P12=25
	P13=26	P14=27	P15=28	P16=29;
*ALGO3:PIDN	P1=11	P2=12	P3=14	P4=15
	P5=7	P6=8	P7=9	P8=2
	P9=16	P10=17	P11=18;	
*ALGO4:OP	P1=1	P2=11	P3=12	P4=1
	P5=3	P6=4	P7=1	P8=2
	P9=22	P10=23	P11=20	P12=21
	P13=13;			
*ALGO5:ALARM	P1=6	P2=7	P3=1;	
*ALGO6:MODE	P1=5	P2=1	P3=1	P4=2
	P5=8;			

```
*FLOAT1:OP1          v=50;
*FLOAT2:PVE1         v=100;
*FLOAT3:SPE1         v=100;
*FLOAT4:CSP1         v=100;
*FLOAT5:EUHI         v=200;
*FLOAT6:EULO         v=0;
*FLOAT7:K            v=3;
*FLOAT8:TINT         v=1;
*FLOAT9:TDER         v=0;
*FLOAT10:TD          v=0;
*FLOAT11:CVPD        v=0;
*FLOAT12:CVI         v=0;
*FLOAT13:OPVIRT1     v=50;
*FLOAT14:PV          v=50;      %
*FLOAT15:SP          v=50;      %
*FLOAT16:LASTD       v=0;
*FLOAT17:LASTP       v=0;
*FLOAT18:LRATE       v=0;       Used in PIDX algorithm only!
*FLOAT20:OPHI        v=95;
*FLOAT21:OPLO        v=5;
*FLOAT22:IHI         v=80       a=1;
*FLOAT23:ILO         v=20;
*FLOAT24:PVHI        v=190;
*FLOAT25:PVLO        v=10;
*FLOAT26:DEVHI       v=20;
*FLOAT27:DEVLO       v=-20;
*FLOAT28:SPHI        v=200;
*FLOAT29:SPLO        v=0;


*STATUS1:MODE1       v=1        s0=ÇMANUALÇ      s1=ÇÇAUTOÇÇ
s2=ÇÇCASCÇÇ                     s3=ÇI-MANÇÇ      s4=ÇI-AUTOÇ
                                s5=ÇI-CASCÇ;
*STATUS2:EQUATION    v=0        s0=ÇTYPEÇAÇ      s1=ÇTYPEÇBÇ
                                s2=ÇTYPEÇCÇ;
*STATUS3:ACTION1     v=1        s0=ÇDIRECTÇ      s1=ÇREVERSE;
*STATUS4:OPCALC1     v=1        s0=ÇÇREALÇÇ      s1=ÇÇVIRTÇÇ;
*STATUS5:INIT1       v=0        s0=ÇÇÇÇÇÇÇÇ      s1=ÇÇINITÇÇ;
*STATUS6:PVALARM1    v=0        s0=ÇÇÇÇÇÇÇÇ      s1=INSTÇHIÇ
                                s2=INSTÇLOÇ
                                s3=ÇROCÇHIÇ      s4=ÇROCÇLOÇ
                                s5=ÇPVÇHIÇÇ      s6=ÇPVÇLOÇÇ
                                s7=ÇDEVÇHIÇ      s8=ÇDEVÇLOÇ;
```

```
*STATUS7:OPALARM1    v=0      s0=ÇÇÇÇÇÇÇÇ   s1=ÇOPÇHIÇÇ
                              s2=ÇOPÇLOÇÇ
                              s3=ÇINTÇHIÇ   s4=ÇINTÇLOÇ;
*STATUS8:CONFIG1     v=0      s0=ÇÇÇÇÇÇÇÇ   s1=ÇÇINITÇÇ
                              s2=TRACKING   s3=ÇIÇ&ÇTRÇ;
*INT1:STATUS1        v=0;
*FLOAT46:LVL-SIM     v=50;
```

K.1.2 Status Word for PID-Block Interaction and General Alarms

External status comes from down stream point and has the same structure as the internal status.

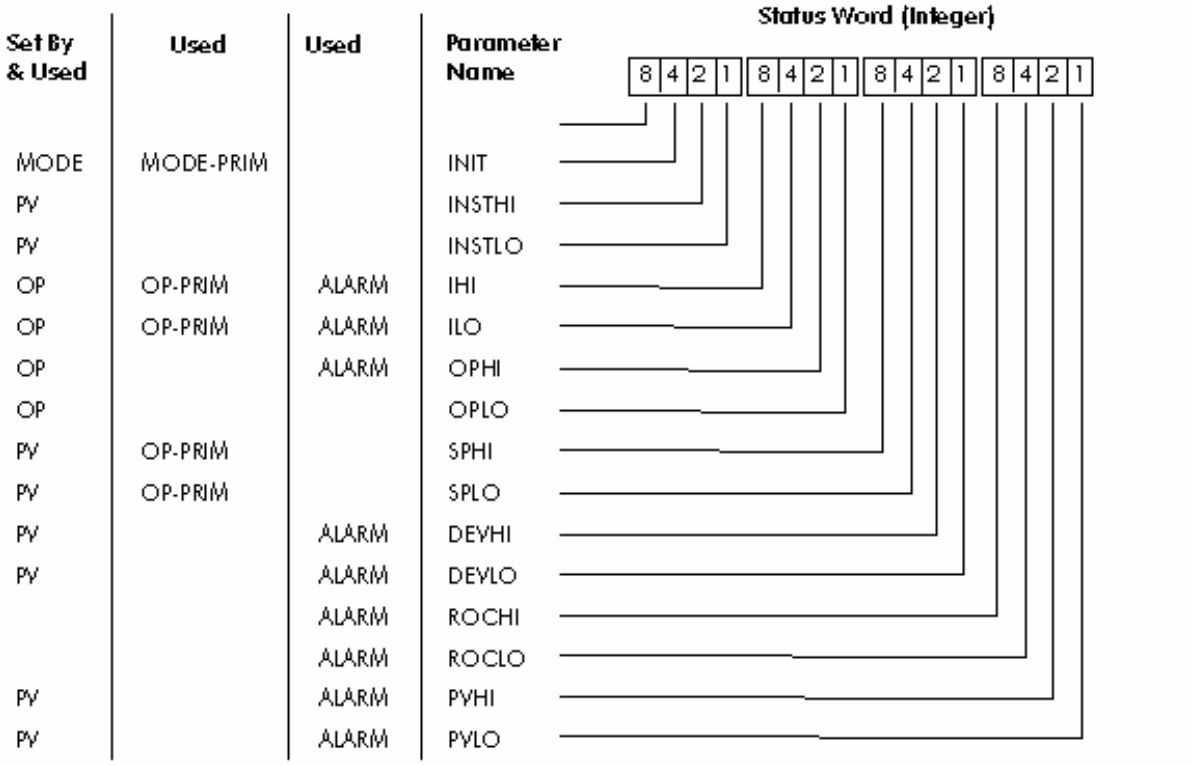


Figure K.2

PIDN

PID-NORMAL (real) Algorithm. This algorithm is best suited as a field controller (secondary with field I/O etc.).

K * (1+sT2)/(1+saT2) * (1 + 1/sT1) a = 1/10.

P1 **CVPD** is the incremental **C**ontrol **V**alue for **PD** control of float type.
P2 **CVI** is the incremental **C**ontrol **V**alue for **I** control of float type.
P3 PV input in % of Range of float type.
P4 SP input in % of Range of float type.
P5 Gain K of float type.
P6 Integral Time Constant T1 of float type.
P7 Deriv. Time Constant T2 of float type.
P8 Equation Type (ET) A, B or C of status type. ET=A for PID on error. ET=B for PI on error and D on PV. ET=C for I on error and PD on PV.
P9 Auxiliary variable of float type to hold previous scan's derivative calculation.
P10 Auxiliary variable of float type to hold previous scan's proportional calculation.

PIDX

PID-NORMAL (ideal) Algorithm. This algorithm is best suited as a high level controller (like computer resident primary controllers), without direct field I/O, using pre-processed data only.

K * (1 + 1/sT1 + sT2)

P1 **CVPD** is the incremental **C**ontrol **V**alue for **PD** control of float type.
P2 **CVI** is the incremental **C**ontrol **V**alue for **I** control of float type.
P3 PV input in % of Range of float type.
P4 SP input in % of Range of float type.
P5 Gain K of float type.
P6 Integral Time Constant T1 of float type.
P7 Deriv. Time Constant T2 of float type.
P8 Equation Type (ET) A, B or C of status type. ET=A for PID on error. ET=B for PI on error and D on PV. ET=C for I on error and PD on PV.
P9 Auxiliary variable of float type to hold previous scan's derivative calculation.
P10 Auxiliary variable of float type to hold previous scan's proportional calculation.
P11 Auxiliary variable of float type to hold previous scan's rate of change calculation.

PV

Provides PV / SP limit calculation, alarm status setting and conversion to % values. This algorithm uses P1 to P16. Calculation should take place before PID.

P1 PV output in % of float type.
P2 SP output in % of float type.
P3 Local SP (LSP) in EU of float type.
P4 PV input in EU of float type.
P5 SP input in EU of float type.
P6 MODE of control (0=MAN, 1=AUTO, 2=CASC, 3=I-MAN, 4=I-AUTO, 5=I-CASC) of status type.
P7 PV-TRACKING (0=OFF, 1=ON) of status type.
P8 STATUS-WORD of status type (see explanation before).
P9 EUHI (Upper range limit) of float type.
P10 EULO (Lower range limit) of float type.
P11 PVHI (PV alarm limit) of float type.
P12 PVLO (PV alarm limit) of float type.
P13 DEVHI (PV-SP alarm limit) of float type.
P14 DEVLO (PV-SP alarm limit) of float type.
P15 SPHI (SP limit) of float type.
P16 SPLO (SP limit) of float type.

OP

Calculates the absolute value based on the increments available from PIDN or PIDX algorithm. Multiple OP-Algorithm blocks may be added to the same PID-Block if multiple OP destinations are required. Initialization of the OP-value takes place automatically if used in cascade mode. Uses P1 to P13.

P1 OP (Control Destination)
P2 **CVPD** is the incremental **C**ontrol **V**alue for **PD** control of float type.
P3 **CVI** is the incremental **C**ontrol **V**alue for **I** control of float type.
P4 MODE of control (0=MAN, 1=AUTO, 2=CASC, 3=I-MAN, 4=I-AUTO, 5=I-CASC) of status type.
P5 REV control action if 1 (0=direct) of status type.
P6 LT is the Limit calculation Type (0=Real / Unsaturated OP-Limits, 1=Virtual / Saturated OP-Limits) of status type
P7 STATUS-WORD of this controller of status type (see explanation before).
P8 External STATUS-WORD of downstream (secondary) controller of status type (see explanation before).

P9 IHL (Integral limit) of float type.
P10 ILL (Integral limit) of float type.
P11 OPHI (OP limit) of float type.
P12 OPLO (OP limit) of float type.
P13 OPVIRT is the virtual, internal and unlimited OP value for saturated OP limit calculation of float type. This parameter has to be provided, independently of the value of parameter LT.

MODE

Monitors and calculates Initialized and non-initialized mode values. It serves for the propagation of INIT between primary and secondary controllers.

P1 INIT for display purpose only of status type.
P2 MODE of control (0=MAN, 1=AUTO, 2=CASC, 3=I-MAN, 4=I-AUTO, 5=I-CASC) of status type.
P3 STATUS-WORD of this controller of status type (see previous explanation).
P4 External STATUS-WORD of downstream (secondary) controller of status type (see previous explanation).
P5 INITIALISATION and PV-TRACKING variable of status type.

ALARM

Provides two Alarm Status Var (PVALARM, OPALARM) for display and general purpose. The two variables are set in accordance with the STATUS-WORD (see explanation before) or any ALARM-STATUS-WORD provided by single alarm algorithms.

P1 PVALARM output (PV alarm status) of status type.
P2 OPALARM output (OP alarm status) of status type.
P3 STATUS-WORD of status type (see explanation before).

K.2 General Algorithms

LEADLAG

First Order Lead-Lag calculation

(1+sT1)/(1+sT2)

P1 Output of float type.
P2 Input of float type.
P3 Lead Time Constant T1 of float type.
P4 Lag Time Constant T2 of float type.
P5 Auxiliary variable of float type to hold previous scan's lag calculation.

SUM

"Summer" to add two inputs: Output = InputA + InputB

P1 Output of float type
P2 InputA of float type
P3 InputB of float type

MUL

"Multiplier" to multiply two inputs: Output = InputA * InputB

P1 Output of float type
P2 InputA of float type
P3 InputB of float type

RATIO

Ratio calc.: Output = Input * Ratio + Bias

P1 Output of float type
P2 Input of float type
P3 Ratio of float type
P4 Bias of float type

PROP

Incremental Proportional calculation: ¡Output = K * ¡Input

P1 Output of float type
P2 Input of float type
P3 K (Gain) of float type
P4 Auxiliary variable of float type to hold previous scan's Output calculation.

INT

Incremental Integral calculation: 1 / sT

P1 Output of float type
P2 Input of float type
P3 T (Integral Time Constant) of float type

DERN

Incremental Derivative calculation (real form): (1+sT)/(1+saT) a = 1 / 10

P1 Output of float type
P2 Input of float type
P3 T (Deriv. Time Constant) of float type
P4 Auxiliary variable of float type to hold previous scan's derivative calculation.

DERX

Incremental Derivative calculation (real form): sT

P1 Output of float type
P2 Input of float type
P3 T (Deriv. Time Constant) of float type
P4 Auxiliary variable of float type to hold previous scan's derivative calculation.

LAG

First Order Lag calculation $1/(1+sT)$

P1 Output of float type.
P2 Input of float type.
P3 Time Constant in minutes of float type.

NOISE

Superimposed noise calculation:
Output = Input + K * RANDOM
whereby RANDOM is noise between -0.5 and +0.5.

P1 Output of float type
P2 Input of float type
P3 K (Noise-Gain) of float type

DISTURB

General Disturbance Generator
RawDist = RawDist + K * RANDOM
Output = (LAG with TC) * RawDisturb
where LAG serves as low-path filter and as return time constant for manual changes of Output.

P1 Output of Disturbance of float type.
P2 K (Disturbance-Gain) of float type.
P3 TC in minutes and of float type.
P4 Raw Disturbance Value of float type.
P5 Disturbance HI-Limit of float type.
P6 Disturbance LO-Limit of float type.
P7 Disturbance BIAS of float type.

SINE

Sine Wave Generator $w/(s^2 + w^2)$

P1 Output of Sine Wave of float type (initialize to value of base line).
P2 Baseline of float type.
P3 Time Constant in minutes of float type.
P4 Auxiliary variable of float type (initialize to first maximum or

SNDORDER

Second Order System with variable damping

$1 / (s^2 + 2\Omega_n s_n^2)$

$w_d = w_n (1 - \Omega d^2)½$

$Tn = 1 / w_n$

P1 Output of float type.
P2 Input of float type.
P3 Damping factor Ω of float type.
P4 Natural Time Constant Tn in minutes of float type.
P5 Auxiliary variable of float type (initialization value of P2 and P5 should be equal).

HILIM

The value of P1 will not exceed the value of P2.

P1 Variable of float type, to be monitored and clamped at limit.
P2 Limit of P1 of float type.

LOLIM

The value of P1 will not go below the value of P2.

P1 Variable of float type, to be monitored and clamped at limit.
P2 Limit of P1 of float type.

ROCHILIM

The value of P1 will not increment by more than the value of P2.

P1 Variable of float type, to be monitored and manipulated by the limit when necessary.
P2 Increment Limit of P1 of float type (Rate of Change per Minute).
P3 Auxiliary variable of float type.

ROCLOLIM

The value of P1 will not decrement by more than the value of P2.

P1 Variable of float type, to be monitored and manipulated by the limit when necessary.
P2 Decrement Limit of P1 of float type (Rate of Change per Minute).
P3 Auxiliary variable of float type.

ILIM

Incremental Integrator with integral limits and integral wind-up handling: 1 / sT

P1 Output of float type
P2 Input of float type
P3 T (Integral Time Constant) of float type
P4 Alarm of status type:
0=No Integral Alarm
1=Integral High, increment suspended
2=Integral Low, decrement suspended.
P5 Integral High Limit of float type.
P6 Integral Low Limit of float type.

HIAL

An alarm will be raised when the value of P2 exceeds the value of P3. P2 is not clamped and can have any value.

P1 Alarm of status type:
0=No Alarm
1=High Alarm
P2 Variable of float type, to be monitored.
P3 Alarm Limit of P1 of float type.

LOAL

An alarm will be raised when the value of P2 is below the value of P3. P2 is not clamped and can have any value.

P1 Alarm of status type:
0=No Alarm
1=Low Alarm
P2 Variable of float type, to be monitored.
P3 Alarm Limit of P1 of float type.

ROCHIAL

An alarm will be raised when the increment of P2 is above the value of P3. The P2 increment is not limited and can have any value.

P1 Alarm of status type:
0=No Alarm
1=ROC High Alarm
P2 Variable of float type, to be monitored.
P3 Increment Alarm Limit of P2 of float type (Rate of Change per Minute).
P4 Auxiliary variable of float type.

ROCLOAL

An alarm will be raised when the decrement of P2 is below the value of P3. The P2 decrement is not limited and can have any value.

P1 Alarm of status type:
0=No Alarm
1=ROC Low Alarm
P2 Variable of float type, to be monitored.
P3 Decrement Alarm Limit of P2 of float type (Rate of Change per Minute).
P4 Auxiliary variable of float type.

STATUS

A general status word will be updated, based on alarm status variables, set by algorithms as shown in table below. The format of this general status word is consistent with the PID controller status word.

P1 General Status Word of integer type.
P2 to P10 Individual variables of status type as shown in table below.

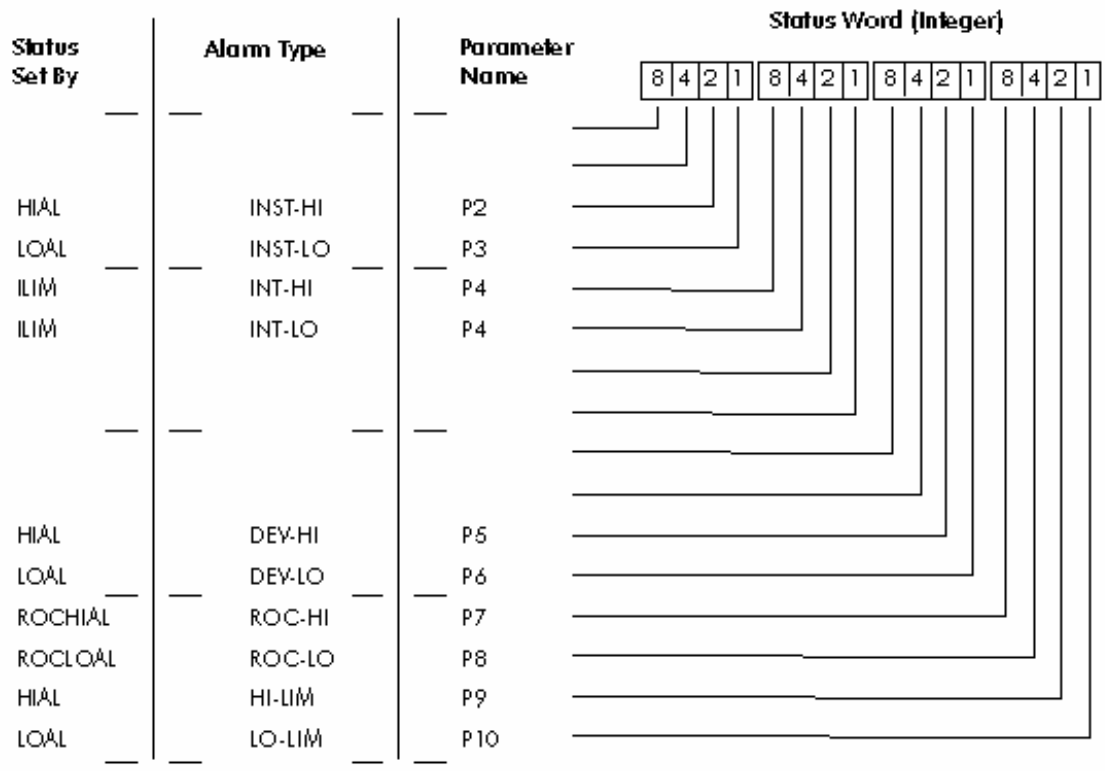


Figure K.3

If no consistency of bit positions for particular alarms is required, they may be used differently, for example according to priorities.

FREQTOTC

Converts a frequency from the frequency domain into the equivalent time constant in the time domain: TC=0.1591549 / FREQ

P1 Output TC of float type
P2 Input FREQ of float type

TCTOFREQ

Converts a time constant from the time domain into the equivalent frequency in the frequency domain: FREQ=0.1591549 / TC

P1 Output FREQ of float type
P2 Input TC of float type

EUTOPCT

Conversion from Engineering Units to %

P1 Output in %
P2 Input in Engineering Units.
P3 100% of range in Engineering Units.
P4 0% of range in Engineering Units.

PCTTOEU

Conversion from % to Engineering Units

P1 Output in Engineering Units.
P2 Input in %
P3 100% of range in Engineering Units.
P4 0% of range in Engineering Units.

LINK

If the function (P3 XOR P3) = TRUE, the Output P1 is set to the value of the Input P2. If the function is NOT TRUE, P1 stays unchanged.

P1 Output of float type.
P2 Input of float type.
P3 Switch variable of status type.
P4 Constant, not related to any type variable:
P4=0 If switch P3 is TRUE, P1 will be set by P2
P4=1 If switch P3 is NOT TRUE, P1 will not be modified at all.

MATH

General Purpose Mathematical Algorithm:

OP=(K1*X1+K2*X2) / (K3*X3+K4*X4)+K5*X5

Any unwanted part of the formula can be rendered ineffective by initialization of Gain (K1, K2 etc.) to a value of 0.

P1 Output of float type.
P2 Input X1 of float type for 1st product.
P3 Gain K1 of float type for 1st product.
P4 Input X2 of float type for 2nd product.
P5 Gain K2 of float type for 2nd product.
P6 Input X3 of float type for 3rd product.
P7 Gain K3 of float type for 3rd product.
P8 Input X4 of float type for 4th product.
P9 Gain K4 of float type for 4th product.
P10 Input X5 of float type for 5th product.
P11 Gain K5 of float type for 5th product.

DIV

Divide algorithm: OP = NOMINATOR / DENOMINATOR

If the absolute value of the DENOMINATOR is less than 0.1% of the NOMINATOR, the OP value is set to BAD.

P1 OP of float type
P2 NOMINATOR of float type
P3 DENOMINATOR of float type

HEATCOMP

Calculates the required FUEL flow for a feed heater to heat up feed material from TIN (feed inlet temperature) to TOUT (feed outlet temperature). Gain K represents a scaling factor, taking into account things such as fuel efficiency etc. FEED is the feed flow, which may vary over time.

FUEL = K * FEED * (TOUT - TIN)

P1 Output FUEL of float type.
P2 FEED of float type.
P3 TOUT of float type.
P4 TIN of float type.
P5 K of float type.

HEATSIM

Calculation for process simulation purposes only. It simulates the feed outlet temperature TOUT of a feed heater. (see HEATCOMP algorithm description above as well)

TOUT = K * FUEL / (FEED + 1) + TIN

P1 Output TOUT of float type.
P2 FUEL of float type.
P3 FEED of float type.
P4 TIN of float type.
P5 K of float type.

MASSFLOW

Massflow compensation, taking into account the volumetric flow (in real life: flow meter value), the temperature and pressure of the material. Gain K represents a general scaling factor, taking into account parameters such as specific mass, different engineering units etc.

MFLOW=K*FLOW*(PRESS / (TEMP-273.15))½

P1 Output MFLOW of float type (massflow).
P2 FLOW of float type (volumetric flow).
P3 PRESS of float type (pressure).
P4 TEMP of float type (temperature).
P5 Gain K of float type.

DEADTIME

General-purpose dead time calculation as used in industrial controllers.

P1 Output of float type.
P2 Input of float type.
P3 Dead time in minutes (float).
P4 Internal pointer variable. This variable itself is an integer type, but its contents is the number of the first of 22 sequential variables of float type. These 22 sequential variables are the dead time history buffer. They have to be reserved for history collection and MUST NOT be used otherwise.
P5 Internal pointer variable. This variable itself is of integer type, but its contents is the number of the float variable holding the oldest history value within the dead time buffer.

Example:

```
*INT23:BUFFIDX          v=132;    dead time buffer: float132 to 153
*FLOAT34:DEADTIME       v=0.3;    minutes
*FLOAT35:INPUT          v=50;
*FLOAT83:OUTPUT         v=50;
*ALGO5:DEADTIME         P1=83     P2=35 P3=34 P4=23 P5=24;
```

The parameters P4 and P5 of the DEADTIME algorithm have to be of integer type. Their purpose is to provide pointers for calculating history data. Parameter P4 contains the index number of the first of 22 sequential floating point variables to be used as dead time history buffer. Parameter P5 contains the index number of the floating point variable containing the oldest history value. No parameter definition for integer 24 is required, since integer 24 is used as an internal parameter of DEADTIME and neither initialization nor display is necessary. There is no parameter definition required for float variables 132 to 153 as well.

K.2.1 General Notes Concerning Algorithms

- **Multiple Use of Variables**

  The same variable may be used within one algorithm several times as different command parameters. This is only possible, if the parameters are of the same type. It may give different results as shown in the following two examples of the NOISE algorithm.

  Example 1:

```
*FLOAT32:PV          v=50;
*FLOAT33:PV-NOISE    v=50;
*FLOAT34:K-NOISE     v=1;
*ALGO2:NOISE         P1=33     P2=32     P3=34;
```

  In this example, assuming that PV (float 32) or PV-NOISE (float 33) are not manipulated, PV-NOISE will stay within the range of 50±0.5. Hence PV-NOISE will not drift away from 50.

  Example 2:

```
*FLOAT32:PV          v=50;
*FLOAT34:K-NOISE     v=1;
*ALGO2:NOISE         P1=32     P2=32     P3=34;
```

  In this example, assuming that PV (float 32), which is used as P1 and P2, is manipulated, then PV will drift away from 50 towards unpredictable values. A trend display of PV will show an unpredictable, non cyclic trend.

- **Incremental Algorithms**

  The incremental algorithms PIDN, PIDX, PROP, INT, DERN, DERX and ILIM may have the same variable (of float type) configured as P1 (Output). If the same variable is jointly used as P1 by two or more of these incremental algorithms, the value of P1 represents the output of an incremental "Summer" of the individual outputs. The following example shows this, using the algorithms PROP and INT to configure a simple PI-Controller without alarm, limit and status handling.

  Example:

```
*FLOAT41:SETPOINT      v=50;
*FLOAT42:PV            v=50;
*FLOAT43:REVERSE       v=-1;     v=-1      for reverse control action
*FLOAT44:PROP-GAIN     v=1.5;
*FLOAT45:ERROR         v=0;
*FLOAT46:INTERNAL-USE  v=50;   internal use of algorithm PROP
*FLOAT47:INT-TC        v=1.5
*FLOAT49:OUTPUT        v=50;

*ALGO1:RATIO           P1=45  P2=42  P3=43     P4=42;
*ALGO2:PROP            P1=49  P2=45  P3=44     P4=46;
*ALGO3:INT             P1=49  P2=45  P3=47
```

  It is important to notice, that both algorithms, PROP and INT use the same variable as their output. The end result in the output variable OUTPUT is an output value which has been incremented (or decremented) by both algorithms. This is in actual fact an incremental "Summer" of proportional and integral control actions.